INSTITUTO FEDERAL DE EDUCAÇÃO, CIÊNCIA E TECNOLOGIA DO NORTE DE MINAS GERAIS- *CAMPUS* ALMENARA

IFNMG

Tecnologia em Análise e Desenvolvimento de Sistemas

CAÍQUE SANTOS CARDOSO

PHARUS: UM SISTEMA WEB PARA GESTÃO DE EVENTOS

Almenara

2021

CAÍQUE SANTOS CARDOSO

PHARUS: UM SISTEMA WEB PARA GESTÃO DE EVENTOS

Trabalho apresentado à disciplina Trabalho de Conclusão de Curso II, do curso de Análise e Desenvolvimento de Sistemas, como pré-requisito para conclusão do mesmo.

Orientador:
Prof. Ms. Marcos Vinícius Montanari

Almenara
2021

C268p Cardoso, Caíque Santos

Pharus: um sistema web para gestão de eventos / Caíque Santos Cardoso – 2021.

46 f. : il. col.
Orientador: Prof. Ms. Marcos Vinícius Montanari
Área de Concentração: Análise de Sistemas.

Trabalho apresentado ao Colegiado do Curso de Graduação em Tecnologia em Análise e Desenvolvimento de Sistemas do Instituto Federal do Norte de Minas Gerais - *Campus* Almenara, como requisito parcial para obtenção o do título de Tecnólogo em Análise e Desenvolvimento de Sistemas.
Inclui bibliografia.

1. Sistema web. 2. Gestão de eventos. 3. Gerenciamento. I. Montanari, Marcos Vinícius. II. Instituto Federal do Norte de Minas Gerais – Campus Almenara. III. Título.

CDD 004.2

Ficha catalográfica elaborada por Lissandra Ruas Lima CRB6 – 2851

Agradecimentos

Agradeço aos meus familiares por todo o apoio que me deram durante toda a jornada da minha graduação e, principalmente, por toda a assistência e confiança durante todo o processo de execução desse trabalho. Agradeço ainda por todo o incentivo que me deram durante toda a minha jornada enquanto aluno.

Agradeço ao professor e orientador Me. Marcos Vinícius Montanari pela amizade, experiência e conhecimentos compartilhados durante todos esses anos em que tive a oportunidade de ser seu aluno e orientando. Agradeço fortemente também ao professor Alan Teixeira de Oliveira, pelas contribuições oferecidas para a execução desse plano.

Agradeço também a Heloísa André Donato, pela sua assistência, contribuições, incentivo e apoio durante todo o planejamento e execução deste trabalho, bem como por toda a nossa convivência, companheirismo e amizade cultivados durante vários anos.

Por fim, agradeço ao Instituto Federal do Norte de Minas Gerais - *Campus* Almenara e a todo o seu corpo docente por todas as oportunidades e conhecimentos passados.

Resumo

Ao se analisar o cenário de realização de eventos acadêmicos do Instituto Federal do Norte de Minas Gerais- *Campus* Almenara, percebe-se a necessidade de um sistema que seja capaz de atender a todas as carências dos eventos que a instituição realiza. Consoante (MARTINS FILHO, 2018) as instituições não estão utilizando soluções definitivas para a gestão de seus eventos, estando a maioria trazendo sistemas pequenos e adaptados. Tendo em vista as considerações apresentadas até aqui, o presente trabalho apresenta o desenvolvimento de um sistema *web* capaz de gerir toda a manutenção dos eventos realizados na instituição. Dentro de suas funcionalidades apresentam-se funções como a capacidade de realizar o controle de presença, manutenção de atividades, gerar de forma automática crachás e certificados, disponibilizar informações organizacionais acerca dos eventos. Os objetivos foram alcançados, levando-se em conta que o sistema é capaz de atender a todos os requisitos propostos durante o processo de engenharia de *software.*

Palavras-chave: sistema web, gestão de eventos, gerenciamento.

SUMÁRIO

1. INTRODUÇÃO 7

2. OBJETIVOS 11

2.1 Objetivo Geral 11

2.2 Objetivos Específicos 11

3. JUSTIFICATIVA 12

4. REFERENCIAL TEÓRICO 15

4.1 PHP 15
4.2 Framework 15
4.3 Model, View, Controller (MVC) 16
4.4 Programação Orientada a Objetos(POO) 18
4.5 Object-Relational Mapping (ORM) 19
4.6 Laravel 19
4.7 HTML 20
4.8 CSS 21
4.9 Bootstrap 22
4.10 Front-end 22
4.11 Back-end 23
4.12 Git 23

5. METODOLOGIA 24
5.1 Engenharia de Software 24
5.2 Design do Software 26
5.2.1 Geral 26
5.2.2 Construção do Front-end 29
5.2.3 Banco de Dados 30
5.2.4 Versionamento 33

6. RESULTADOS E DISCUSSÕES 35
6.1 Das funcionalidades 35
6.2 Telas 38

7. TRABALHOS SEMELHANTES 43

8. CONSIDERAÇÕES FINAIS 45
8.1 Trabalhos futuros 45

REFERÊNCIAS 46

1. INTRODUÇÃO

A Tecnologia da Informação (TI) vem ganhando mais espaço, estando presente em praticamente todos os cenários. Instituições como o Instituto Federal do Norte de Minas Gerais (IFNMG), que possuem diversas, vêm cada vez mais fazendo o uso da TI e Sistemas de Informação para gerência e controle de suas atividades. Tendo em vista que o desenvolvimento da tecnologia aumenta cada vez mais o fluxo de dados e, por consequência, a utilização de sistemas informatizados é quase uma necessidade natural para as organizações, considerando que os sistemas integrados são reconhecidos como instrumentos viáveis, como aponta Cavalcante Neto (2000).

A dita instituição já concentra em seu ecossistema alguns sistemas de informação para apoio de suas atividades. Dentre esses sistemas podem ser mencionados o próprio portal institucional, que inclui informações sobre toda a rede do IFNMG e o sistema de estágio. Tais ferramentas possuem como característica principal serem facilitadores do acesso à informação, tanto para a comunidade externa e interna, e trazer contribuições que visam facilitar a gestão de atividades organizacionais.

Todo esse contexto está envolto a necessidade de se trabalhar com a informação, tendo em vista que essa possui grande importância para as organizações que por meio de seu tratamento correto permite maior facilidade na tomada de decisões e planejamento estratégico, uma vez que:

> A informação, quando adequadamente assimilada, produz conhecimento, modifica o estoque mental de informações do indivíduo e traz benefícios ao seu desenvolvimento e ao desenvolvimento da sociedade em que ele vive. (BARRETO, 1999).

Uma das atividades mais recorrentes em instituições acadêmicas é a realização de eventos científicos/acadêmicos. Tais eventos apresentam demandas específicas devido ao seu porte e finalidade, o que acaba por exigir a utilização de sistemas de gerência que possam atender a todas as demandas da instituição no

contexto dos eventos que realiza, esse apontamento vem em conformidade com os pensamentos de Martins Filho e Glanzmann (2018) pois afirmam que:

> As Instituições Educacionais nos dias de hoje vêm trazendo uma demanda cada vez maior por eventos sejam eles congressos, semanas de inovação e pesquisa, ou mesmo eventos voltados para algum curso específico. Existe cada vez mais a necessidade de se hospedar e gerenciar eventos de variados portes. (MARTINS FILHO e GLANZMANN, 2018).

Em meio a esse cenário percebe-se que as instituições muitas vezes não possuem uma solução específica e definitiva, consoante Martins Filho e Glanzmann(2018), para atender as demandas de seus próprios eventos, que podem incluir aspectos como:

- Quantidade de participantes
- Gerar relatórios
- Requisitos não funcionais com características específicas desejadas pela instituição
- Requisitos funcionais com características específicas desejadas pela instituição
- Regras de neǵocio específicas
- Acesso à informação
- Disponibilidade da informação
- Validação e controle de acesso em eventos *online*
- Liberdade para definição de eventos de grande porte

Tais demandas são levantadas pela própria instituição, por meio de seus membros que são responsáveis por coordenar a realização dos eventos. Vale salientar que alguns dos aspectos mencionados podem ser encontrados em soluções já disponíveis, obtidas a partir do levantamento de trabalhos correlatos, entretanto, a utilização destas ferramentas muitas vezes não é de uso gratuito. Dessa forma, as instituições acabam por ter que recorrer a sistemas externos gratuitos, ou mesmo pagos, que ainda não conseguem atender a toda a demanda exigida, uma vez que cada instituição e cada evento possuem as suas particularidades.

Haja vista as considerações até aqui apresentadas, percebe-se então a necessidade de construção de um sistema que seja capaz de facilitar o processo de gerenciamento dos eventos realizados no IFNMG - *Campus* Almenara. A construção desse *software* se apresenta como imprescindível, pois possibilitará que os eventos do IFNMG - *Campus* Almenara possam ser geridos por um ferramenta própria da instituição, que por ser desenvolvida sob os moldes desejados pelo *campus,* possibilita atender uma gama maior de demandas, permite a eliminação de gastos com o desenvolvimento e utilização do sistema e potencializa maior facilidade no acesso da informação. Observa-se ainda a garantia de implementação de novas funcionalidades, caso novas carências possam surgir.

Assim sendo, esse trabalho apresenta a proposta e construção de um sistema de gestão de eventos científico/acadêmicos no IFNMG - Almenara. Assim, entende-se que o objetivo principal deste trabalho consiste na construção de um sistema para gerir os eventos realizados pelo *campus* Almenara. Para tal, é preciso levantar os requisitos do sistema, determinar as tecnologias que serão utilizadas e definir as necessidades e apontamentos principais dos eventos.

O sistema deve ser capaz de atender a todos os requisitos da instituição, hospedar os eventos que serão realizados e facilitar o acesso às informações que estão relacionadas ao cenário dos eventos, tanto a nível organizacional, quanto ao nível externo, apenas de participação, deve ser capaz de gerir eventos de grande, médio e pequeno porte. O *software* deve ainda ser seguro, de fácil utilização e possuir garantia de disponibilidade.

Vale ainda evidenciar que outros trabalhos com a mesma finalidade já foram desenvolvidos, fator esse que demonstra mais uma vez a carência que as instituições acadêmicas apresentam frente a utilização de *softwares* para gerir seus próprios eventos. Dentre os trabalhos relacionados podem ser citados:

- Academeek: Sistema voltado para gestão de eventos gerais.
- e-Vent-Br: Sistema voltado para gestão de eventos gerais, mas diferencia-se por ser construído sob uma licença *open source*.
- S-Eva: o S-Eva é um protótipo de sistema desenvolvido para gerenciar eventos acadêmicos, mas ainda não atingiu o estágio de maturidade

para ser posto em produção.

As soluções propostas em trabalhos semelhantes (que serão abordadas de forma mais aprofundada no item Trabalhos Correlatos), embora de extrema relevância, não se apresentam como uma solução viável, pois cada uma delas apresenta ainda carências, levantadas ao realizar o levantamento dos trabalhos, dada a realidade já apresentada de demandas específicas da instituição. Salienta-se ainda que usando ferramentas disponibilizadas por terceiros ainda se tem a defasagem de soluções próprias e definitivas apontada por Martins Filho e Glanzmann (2018).

2. OBJETIVOS

2.1 Objetivo Geral

Desenvolver um Sistema Web/Mobile responsável pela gerência dos eventos realizados pelo IFNMG-Campus Almenara.

2.2 Objetivos Específicos

- Levantar os requisitos do Sistema.
- Determinar as tecnologias que serão utilizadas.
- Definir as necessidades e apontamentos principais dos eventos.
- Definir qual o tipo de sistema será desenvolvido(Web, Desktop, Mobile).
- Realizar o levantamento de trabalhos semelhantes.

3. JUSTIFICATIVA

A construção do software justifica-se pela necessidade de um sistema próprio que seja capaz de gerenciar os eventos realizados pelo IFNMG- Almenara, uma vez que um sistema com tais características não exige a necessidade de recursos financeiros e, por ser desenvolvido especificamente para a instituição, sua feitura tem atenção especial para atender as demandas que os eventos da instituição exigem. Observa-se também que:

> *Instituições Educacionais não vêm utilizando nenhuma solução definitiva para facilitar a gerência desses eventos, a maior parte trás sistemas pequenos e adaptados ano após ano e que muita das vezes acarreta muito trabalho para os setores que organizam o evento.*" (MARTINS FILHO, GLANZMANN, 2018).

Deve-se ainda levar em consideração a melhora na facilidade de acesso, garantia de serviço e padronização.

A utilização de um sistema de gerência único permite maior facilitação na compreensão de dados gerais, uma vez que a tecnologia da informação proporciona novas infraestruturas em diversas atividades na vida organizacional, como aponta (ZUBOFF, 1994), o que permite à instituição maior apoio na tomada de decisões e melhorias na realização de eventos futuros.

Com o desenvolvimento do sistema, torna-se mais fácil aos participantes consultarem informações referentes a eventos passados dos quais obtiveram certificados de participação tanto no evento, quanto em atividades, tais como minicursos, palestras, oficinas e rodas de conversa. Esse é um fator de grande importância, uma vez que os certificados são de vital importância para composição do currículo e de horas complementares.

Deve-se ainda considerar a importância dos sistemas de automação nos processos organizacionais, observando o ganho de tempo proporcionado pela utilização dos mesmos, salientando que "*À medida que as organizações aplicam a tecnologia da informação, elas tendem a desenvolver mecanismos que permitem que informações sejam automaticamente geradas e captadas*" (ZUBOFF, 1994).

Outro aspecto que deve ser considerado é a utilização de sistemas de terceiros. Entretanto, é necessário refletir que a utilização de tais mecanismo não é por si só capaz de sanar as carências que são sentidas pela instituição ao promover os seus eventos. A partir do levantamento de trabalhos correlatos, algumas soluções baseadas na criação de *softwares* de gerenciamento puderam ser encontradas, entretanto, mesmo estas soluções ainda não apresentam funcionalidades capazes de suprir as demandas exigidas. Dentre esses aspectos, pode se considerar como exemplos:

- Sistemas incapazes de gerir eventos online, pensando nessa gestão como o tratamento do controle de presença e acesso.
- Sistemas que ainda não atingiram grau de maturidade suficiente para serem postos em ambiente de produção.
- Sistemas que apresentam uma pequena estrutura, dessa forma não são capazes de gerir eventos de qualquer porte.

Os aspectos listados foram percebidos no levantamento de trabalhos semelhantes, que serão apresentados posteriormente. Vale ainda lembrar que ao se utilizar uma solução já existente, os requisitos da instituição acabam por não serem atendidos. Em conformidade com Martins Filho e Glanzmann (2018), é possível concluir que usar ferramentas externas e incompletas não traz viabilidade.

Outro ponto que justifica o desenvolvimento deste sistema está envolto na necessidade de realização de eventos *online*. Com a pandemia do *Sars-Cov-2* ficou evidente a necessidade de realizar eventos com essa classificação, acordante a Valerin e De Sousa (2020), que apontam que por conta da pandemia de covid-19, fato que forçou o distanciamento social como melhor prática de prevenção, juntamente ao cancelamento de eventos com aglomeração obrigou que fossem evidenciados novos meios de acesso à informação. Assim sendo, nada mais viável que a possibilidade de realização de eventos acadêmicos de forma completamente *online.* Tendo em vista tal apontamento, o sistema Pharus deve ser capaz de permitir o gerenciamento de eventos realizados de forma inteiramente *online.*

Portanto, esse sistema possibilita que as comissões organizadoras dos eventos atinjam maior agilidade de trabalho, maior comodidade para o gestor, bem como irá viabilizar para a instituição maior controle e garantia de integridade da informação, que efetivamente contribui para a tomada de decisão a nível organizacional. Salientando as contribuições para eventuais cenários de realização de eventos inteiramente *online.*

4. REFERENCIAL TEÓRICO

Para compreensão geral do sistema desenvolvido, torna-se necessária a apresentação de alguns conceitos que estão presentes no contexto de desenvolvimento do sistema Pharus e do ambiente *web* como um todo. Assim, os seguintes conceitos serão apresentados.

4.1 PHP

PHP, acrônimo recursivo para *Hypertext Preprocessor* (Originalmente chamado de *Personal Home Page*) é uma linguagem de programação voltada para o ambiente *web*. O PHP faz parte do subconjunto de linguagens de script, consoante PHP(2021), entretanto, diferencia-se por ser uma linguagem mais comumente voltada à comunicação no lado do servidor. Assim, o php executa executa os *scripts* presentes na estrutura do código e isso permite maior comunicação entre as páginas, tornando as páginas *web* mais dinâmicas, consoante Rocha *et al* (2016, apud LOBO, 2007, p. 96).

Essa linguagem dispõe de diversos *frameworks* disponibilizados de forma inteiramente gratuita e, com isso, torna-se uma candidata natural a ser utilizada em trabalhos desse cunho e principalmente empregada neste trabalho (ainda que de forma indireta). Em conformidade com Arêas (2011), o PHP possui frameworks muito bons que já se apresentam em fase de maturidade, mas ainda em aperfeiçoamento.

O próximo conceito a ser abordado consiste em ferramentas que buscam otimizar o processo de desenvolvimento de *software*.

4.2 Framework

Durante o processo de evolução da Engenharia de Software, novas práticas, técnicas e ferramentas de desenvolvimento foram surgindo. Nesse processo surgiram os *Frameworks,* poderosas ferramentas de desenvolvimento que visam possibilitar um ganho de produtividade aos desenvolvedores.

Os *Frameworks* caracterizam-se como uma abstração que possibilita a

unificação de códigos comuns entre projetos distintos, dessa forma tornando seu uso muito abrangente e ao mesmo tempo personalizável, uma vez que configurações específicas podem ser aplicadas de acordo com a necessidade do *software* projetado. Assim sendo, *Frameworks* se apresentam como um instrumento capaz de prover estruturas comuns e reutilizáveis para diversos cenários.

Portanto, "*framework nada mais é do que um tipo especial de aplicativo, que oferece um conjunto básico de ferramentas e subsistemas que automatizam grande parte dos serviços que precisam ser implantados nos sistemas para usuários finais*" (2007, p. 236 MELE e NASCIMENTO apud ARÊAS, 2011 p. 20). Com tais ferramentas disponibilizadas pelos *frameworks* o processo de desenvolvimento se torna mais ágil e prático, pois:

> Desta forma, a utilização de frameworks na criação de websites, fornecem uma base para a formatação de elementos comuns e otimizam o trabalho repetitivo, apresentando normalmente definições de formatação com os elementos mais comuns de uma página como por exemplo, cabeçalhos, menus, formulários, tipografia e imagens. (PORTELA, 2016).

Uma vez que os *frameworks* provêm componentes já criados e devidamente configurados, tal fator se apresenta como uma vantagem ao se utilizar essas tecnologias, haja vista que "*Se o desenvolvedor optar pela utilização de um framework, não precisará ter o desgaste de criar tais componentes, sendo que estes estarão sujeitos a muitas falhas" (*ARÊAS, 2011).

Uma vez que se define a utilização de uma determinada tecnologia, deve-se pensar também na estrutura do projeto, dessa forma, o próximo tópico conceitua o modelo estrutural utilizado no sistema Pharus.

4.3 Model, View, Controller (MVC)

Model, *View, Controller*, doravante MVC, é um padrão arquitetural que foi apresentado pela primeira vez em 1978 na linguagem Smalltalk-76, e conforme Rocha *et al* (2016), a utilização do modelo MVC implica que a implementação do software projetado deve ser feita seguindo o paradigma de programação orientada a objetos. Esse padrão consiste na separação dos componentes da aplicação, sendo

eles:

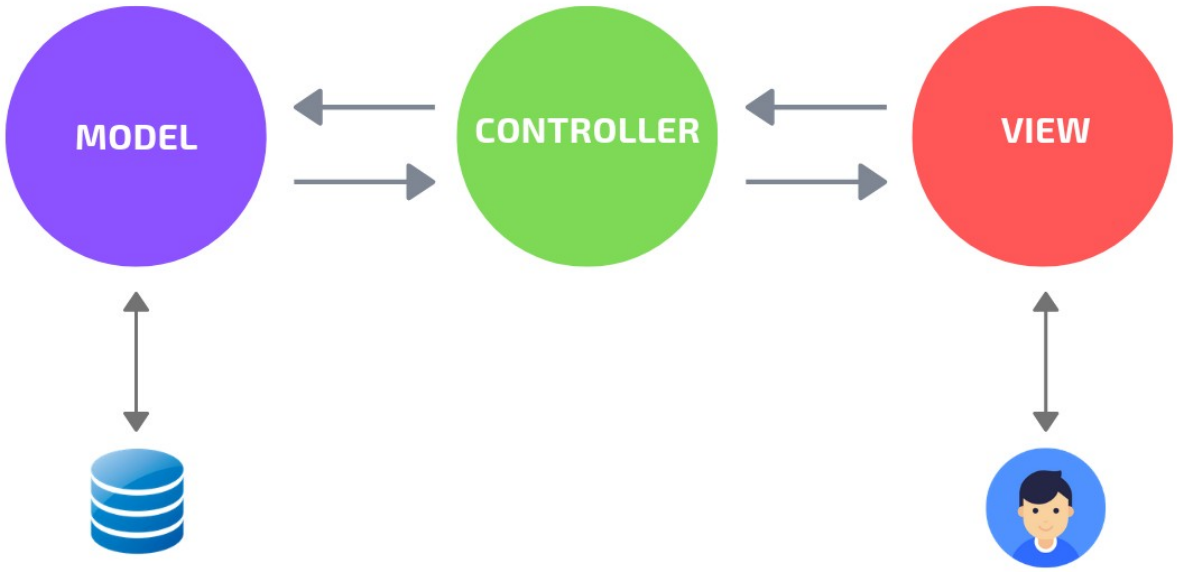
- O *Model,* camada responsável pela comunicação/modelagem dos dados. "*É responsável por garantir a integridade dos dados, o modelo de relacionamento entre entidades e os atributos que compõem cada uma delas*" (ASTOLFI e CABRINI, 2016). Observa-se ainda que:

> A camada de modelo inclui classes exclusivamente responsáveis por intermediar a comunicação da camada controle e as tabelas do SGBD. Portanto, nenhuma camada pode acessar diretamente o banco de dados, a não ser por meio da camada modelo (ROCHA *et al*, 2016).

- O *View*, camada responsável pela comunicação direta com o usuário (pode ser entendida como a camada de apresentação da aplicação), pois como aponta Rocha *et al* (2016), "*A camada de visualização é a parte responsável em interagir como o usuário (front-end)"*.
- *O Controller*, responsável por trabalhar as regras de negócio do sistema e comunicar as duas camadas anteriores. Dessa forma é certo dizer que é nessa camada que as funcionalidades do sistema são implementadas, conforme aponta Astolfi e Cabrini (2016).

Na imagem a seguir é possível observar e compreender a comunicação entre as camadas do modelo MVC.

Figura 1: Representação do funcionamento dos componentes no modelo MVC.



Fonte: (Barbosa, 2019)

Naturalmente, é necessário definir a utilização de um paradigma de programação, para tal, o próximo tópico aborda o conceito de Paradigma de Orientação a Objetos.

4.4 Programação Orientada a Objetos(POO)

Programação Orientada a Objetos, doravante POO, é um paradigma de programação que busca abstrair elementos do mundo para estruturação dos códigos, assim buscando criar uma modelagem mais natural para a resolução dos problemas propostos.

De modo geral, a POO se baseia em dois conceitos básicos, as “Classes” e os “Objetos”, sendo a classe, como o nome sugere, uma forma de classificar elementos que possuem características comuns, enquanto o objeto nada mais é que um elemento pertencente a uma classe. Em outras palavras, a classe pode ser entendida como um molde usado para a obtenção de objetos, assim, os objetos tornam a classe, conceito abstrato, em algo concreto.

Classe, segundo Ricarte (2001) é um gabarito de para a definição de objetos. Ainda consoante o autor, é através da definição de uma classe que se descreve as propriedades que objeto terá. É na classe que os os comportamentos da classe são definidos, tais comportamentos são chamados de métodos. Segundo Ricarte (2001), um método é o equivalente a um procedimento ou uma função e se diferencia por trabalhar apenas a manipulação de suas variáveis locais e os atributos da classe.

Ainda dentro da Orientação a objetos tem-se o conceito de Herança. É o conceito de herança que torna única a Orientação a Objetos, a herança consiste de um mecanismo que possibilita que características comuns a várias classes possam ser compartilhadas em diversas outras classes, assim “*Cada classe derivada ou subclasse apresenta as características (estrutura e métodos) da classe base e acrescenta a elas o que for definido de particularidade para ela.*” (RICARTE, 2001).

O próximo tópico conceitua o mecanismo que busca otimizar a comunicação entre o sistema e o banco de dados do sistema.

4.5 Object-Relational Mapping (ORM)

O termo Object-Relational Mapping, em tradução livre Mapeamento de Objeto Relacional, consiste em um técnica que busca mapear o modelo de objetos desenvolvido na aplicação e o modelo relacional do banco de dados, criando uma espécie de camada de ligação entre os dois, pois como aponta ARÊAS(2011), "*A tarefa primária de um ORM é a tradução transparente entre objetos PHP e linhas de dados relacionais*". Por meio do ORM é possível trazer uma maior aproximação entre a Programação Orientada a Objetos com o Banco de Dados. Ao se fazer uso destas ferramentas é possível diminuir, ou até mesmo dispensar por completo, a escrita de códigos SQL (Structured Query Language).

Entre alguns exemplos de ORMs tem-se:

- Eloquent: Consoante Laravel (2021), o Eloquent é um ORM que torna mais agradável a interação com o banco de dados. Ainda em conformidade com Laravel (2021), ao se utilizar o Eloquent cada tabela do banco de dados recebe um "Modelo" equivalente e por meio desse modelo é feita interação com a tabela;
- Doctrine: De acordo ARÊAS (2011), é um mapeador objeto relacional para PHP, localizado em uma camada de abstração de banco de dados, fornecendo persistência transparente para objetos PHP.

Uma vez que já foram abordados o conceito de PHP e de Framework, o próximo tópico conceitua a tecnologia principal utilizada para o desenvolvimento da aplicação, um *framework* fundamentado na linguagem de programação PHP.

4.6 Laravel

Laravel é um *framework*, fundamentado na linguagem de programação PHP, voltado ao desenvolvimento *web* e estruturado na arquitetura MVC. "*O Laravel é um framework de aplicação web com sintaxe expressiva e elegante*" (Laravel, 2020),

possuindo uma estrutura progressiva que permite a criação de sistemas que podem variar de simples a robustos. Por possuir uma vasta biblioteca de documentação, facilita o aprendizado de novos desenvolvedores sem gerar grande sobrecarga. Este *framework* possui ainda uma estrutura escalável, fundamentada no escalonamento nativo do PHP e no próprio suporte embutido do Laravel voltado para sistemas de *cache*. Assim, "*os aplicativos Laravel foram facilmente escalados para lidar com centenas de milhões de solicitações por mês" (*Laravel, 2020).

Os tópicos seguintes conceituam as tecnologias fundamentais utilizadas no desenvolvimento *web*, sendo a primeira delas o HTML.

4.7 HTML

O HTML (em tradução, Linguagem de Marcação de Hipertexto) é a tecnologia base do cenário de desenvolvimento abordado até aqui. Com o HTML é possível estruturar os elementos das páginas que serão vistas pelo usuário durante sua navegação no meio web. Consoante a De Souza (2016), inicialmente o HTML foi desenvolvido para que pudesse descrever de forma semântica documentos científicos. Sob esse aspecto, o autor ainda afirma que o HTML "*Hoje em dia é uma tecnologia chave da Internet, sendo uma linguagem para estruturação, apresentação e conteúdo para a World Wide Web (WWW). " (*DE SOUSA, 2016).

Por meio do HTML é possível realizar a construção dos *links* de comunicação interna e externa da página, observando aqui que os *links* são partes fundamentais e imprescindíveis da *web*, salientando que todo esse processo de "linkagem" está atrelado ao termo "Hipertexto" presente na própria identificação da linguagem.

Já a marcação se refere às anotações feitas pelo HTML para estruturar e dividir o conteúdo das páginas que são apresentadas por intermédio de um navegador *web*. Essa divisão é feita por meio do uso de "elementos", tais como: head, body, div, a e img. Os elementos são identificados e diferenciados de demais informações textuais mediante o uso de "tags", representadas pela utilização de colchete angular, onde o nome do elemento é apresentado dentro dos colchetes, por exemplo: <body>.

Como aponta De Souza, o HTML, considerando aqui o HTML5, possui

atualmente como principais características:

- Elemento canvas para desenho
- Elementos de vídeo e áudio para reprodução multimídia
- Melhoria no suporte para armazenamento local
- Novos elementos de conteúdo específico
- Novos controles para formulário
- Suporte ao CSS3

O próximo tópico aborda a tecnologia utilizada para se dar estilo as páginas *web.*

4.8 CSS

O CSS, *Cascading Style Sheets* ou Folhas de Estilo em Cascata, é uma linguagem de estilização utilizada para personalizar e, como sua definição sugere, dar estilo às páginas estruturadas em HTML. Acordante a Rabetti (2019), o CSS é a linguagem que define a forma pela qual os documentos serão apresentados em uma página *web.* Ainda consoante ao autor "*É no CSS em que se define várias características visuais que um elemento pode possuir, como sua altura, largura, cor e seu posicionamento, entre outras*", (RABETTI, 2019).

Assim como o HTML, o CSS é um dos pilares do desenvolvimento *web* e possui sua padronização seguindo as especificações da W3C, organização responsável pela padronização da *World Wide Web*.

Para a construção do CSS é necessário a escrita de um arquivo *.css,* uso da *tag* <style></style> ou do atributo "style" dentro de uma *tag*. Para que as configurações sejam aplicadas, é necessário a presença de um seletor (Podendo ser um id, uma div ou a própria *tag),* uma propriedade e um valor. Assim, como explica Rabetti (2019), o CSS pode ser entendido como uma linguagem de busca de seletores, com a finalidade de aplicar de associar o estilo desejado.

O tópico seguinte define um *framework,* fundamentado na linguagem de estilo CSS, utilizado também, naturalmente, para a estilização das páginas *web.*

4.9 Bootstrap

Bootstrap é um *framework* fundamentado na linguagem CSS e tem como objetivo facilitar o processo de criação da estilização dos componentes de interface das páginas *web*. Do acordo Portela (2016), "*O Bootstrap é um conjunto gratuito de ferramentas, é um framework front-end que foi desenvolvido pela equipe do Twitter6 com o intuído de facilitar o desenvolvimento de qualquer site e sistema web.*". Ainda conforme Portela (2016), o Bootstrap é focado nos padrões da W3C e possui compatibilidade com o HTML5 e CSS3.

Esse *framework* possui em sua estrutura a inclusão de diversas classes já pré-configuradas na linguagem de estilo, proporcionando maior agilidade para construção das páginas, em conformidade com Portela (2016), que em seu apontamento afirma que "*Como toda a estrutura do CSS já vem previamente definida, basta procurar o componente que é necessário e adicionar o código devido, em poucos minutos, o desenvolvedor terá um layout pronto para uso*".

Tal agilidade de desenvolvimento pode ser vista como um fator de grande importância quando se tem um cenário de desenvolvimento com uma equipe pequena, onde não há parte responsável por trabalhar o especificamente o *Front-end* da aplicação (conceito que ainda será abordado).

O tópico seguinte conceitua o parte da interação com o usuário, ou seja, todas as páginas do sistema.

4.10 Front-end

O termo *Front-end* está relacionado a toda a parte visual de uma aplicação, na qual é gerado algum tipo de interação com o usuário, consoante Mario Souto (2019), que diz que *front-end* é a parte em que se pode classificar como a parte visual de um *site*. Dessa forma o *Front-end* define tudo que está relacionado a parte gráfica do software e toda essa parte gráfica é criada a partir da utilização de códigos escritos, geralmente, nas linguagens fundamentais do desenvolvimento

web, sendo elas, HTML, CSS e Javascript, bem como a utilização de *frameworks* e bibliotecas fundamentados a partir destas mesmas tecnologias.

O tópico seguinte conceitua a parte que guarda as regras de neǵocio do sistema, que não estão acessíveis ao usuário. com o usuário, ou seja, todas as páginas do sistema.

4.11 Back-end

O termo *Back-end* é, naturalmente, o oposto do *Front-end.* Enquanto o *Front-end* define “a parte da frente” de uma aplicação, o *Back-end* define o que está atrás, ou seja, tudo que não está disponível aos utilizadores. É no *Back-end* que todas as regras de negócio do sistema projetado ficarão armazenadas e onde se encontra a ponte que comunica os dados dos usuários entre o *Front-end* e o Banco de Dados do sistema. Assim, como afirma Mario Souto (2019), o *Back-end* geralmente controla o fluxo de dados entre o *Front-end* e o *Back-end,* aplicando todas as devidas regras de negócio.

O tópico seguinte está relacionado ao versionamento do sistema, para compressão e definição do versionamento será abordado o conceito de Git, sendo esse um sistema para controle das versões.

4.12 Git

O Git consiste em um sistema de versionamento que permite que arquivos possam ser versionados e compartilhados por diversos desenvolvedores, e como aponta Konnorate et al (2019), “*Não podemos deixar de mencionar que o Git é um sistema para controle de versão “distribuído”, ou seja, não depende de um servidor centralizado.*”. Segundo Git (2019), o Git é um sistema de controle de código aberto que foi ideado para ser capaz de lidar com projetos que variam desde de simples a complexos.

5. METODOLOGIA

O trabalho desenvolvido caracteriza-se como uma Pesquisa Aplicada e metodologicamente seguiu algumas etapas que podem ser caracterizadas em dois grupos definidos pelo autor, sendo eles: Engenharia de Software e Design de Software.

A primeira etapa, Engenharia de Software, consiste no levantamento da parte teórica de desenvolvimento de um *software*, assim, está relacionada ao levantamento dos requisitos, definição da metodologia de desenvolvimento, escolha de tecnologias e prazos.

A segunda etapa, Design de Software, consiste efetivamente no processo de construção do sistema, diretamente na parte de codificação e testes. Nessa etapa os releases do sistema foram sendo construídos, testados e validados.

5.1 Engenharia de Software

Para efetuar o desenvolvimento do sistema apresentado neste trabalho, o primeiro passo a ser realizado consistiu no levantamento dos requisitos do sistema. Esse levantamento incluiu os requisitos gerais do software, uma vez que o sistema foi desenvolvido inteiramente seguindo o modelo incremental de desenvolvimento de software, pois é sabido que ao se utilizar metodologias ágeis "*existem maiores possibilidades de atender aos requisitos do cliente, que muitas vezes são mutáveis.*" (DOS SANTOS SOARES, 2004). Para tal, o levantamento dos requisitos foi realizado por meio de entrevistas com os membros da comissão organizadora responsável pela realização do evento "Semana Nacional de Ciência e Tecnologia" na edição realizada no ano de 2019, assim como membros de comissões organizadoras de outros eventos realizados pela instituição, uma vez que o sistema deve ser capaz de gerir todos os eventos que virão a ser realizados pelo *campus*.

As entrevistas realizadas foram orientadas a partir de um grupo de perguntas norteadoras (meio pelo qual propiciou um surgimento de novos questionamentos), assim, classificando-se como uma entrevista semi-estruturada. Durante as entrevistas, foram abordados aspectos gerais de problemas e necessidades de

utilização. Dessa forma toda a conversa foi orientada com a temática a partir das seguintes proposições:

- Quais as ferramentas utilizadas atualmente?
- Quais as principais dificuldades ao se utilizar as ferramentas atuais?
- Quais funcionalidades presentes nas soluções atuais conseguem atender ao esperado?
- Quais funcionalidades presentes nas soluções atuais devem ser mantidas?
- Quais funcionalidades presentes nas soluções atuais não apresentam relevância para o sistema?
- Quais funcionalidades poderiam ser acrescentadas nas soluções atuais?
- Quais funcionalidades o novo sistema deve ter?
- Quais tipos de usuários o novo sistema deve suportar?
- Quais as vantagens presentes nas soluções atuais?
- Quais as desvantagens presentes nas soluções atuais?
- O que poderia ser feito para sanar as dificuldades?
- Quais são as funções que o sistema deve disponibilizar para cada tipo de usuário?
- Quais dados devem ser registrados?
- Qual a melhor plataforma para acesso?
- Quais arquivos o sistema deve ser capaz de gerar?
- Quais informações devem ser customizáveis?
- Por intermédio do sistema, como a comissão organizadora deve se comunicar com os participantes?

Embora no desenvolvimento do *software* não tenham sido utilizadas metodologias ágeis comuns, optou-se pelo uso de uma metodologia mais próxima do *Extreme Programming*(XP), metodologia que assume o desenvolvimento cíclico, onde reuniões são realizadas e nessas reuniões, com cliente presente, requisitos são levantados, um *release* do sistema é apresentado e as refatorações são propostas. A metodologia adotada se diferencia do XP, por não seguir precisamente todas as etapas e conceitos propostos pela *Extreme Programing,* a exemplo: As interações (que no XP geralmente correspondem a uma ou duas semanas) e a

programação em par. Vale salientar que não optou-se pela utilização de uma metodologia comum devido à aspectos externos que impossibilitaram o cumprimento de todas as etapas que são propostas pela metodologia. Dentre esses aspectos podem ser citados: a impossibilidade de realização de reuniões presenciais, a ausência de uma equipe de desenvolvimento e o cenário ao qual o desenvolvimento do sistema se encontra.

Para a construção do sistema o processo de engenharia de *software* se deu por meio de comunicação contínua realizada em sua maioria por meios eletrônicos, devido a facilidade de comunicação que estes meios proporcionam e também pela pandemia do *Sars-Cov-2*, fator que dificultou a realização de reuniões e encontros presenciais, haja vista a necessidade de se manter o distanciamento social para efetividade da medidas de prevenção definidas pelas órgãos de saúde, em conformidade com Aquino *et al* (2020), pois aponta que "*Recomenda-se a implementação de medidas de distanciamento social e de políticas de proteção social para garantir a sustentabilidade dessas medidas*"(Aquino *et al,* 2020). Assim, as funcionalidades foram desenvolvidas periodicamente seguindo a ordem de prioridade definida pelo autor deste trabalho.

A etapa seguinte corresponde ao processo de desenvolvimento do sistema (Entende-se aqui como desenvolvimento e codificação e processos relacionados diretamente ao código fonte). Para essa etapa alguns outros grupos também devem ser destacados, abordando, portanto, as tecnologias utilizadas em cada camada da aplicação.

5.2 Design do Software

5.2.1 Geral

Optou-se pela construção de um sistema *web* devido a facilidade de acesso que sistemas nessa classificação permitem. Tal facilidade pode ser justificada pelo número crescente de pessoas que dispõem do uso de *Smartphones* com acesso a internet, e, com as tecnologias disponíveis nos dias atuais (tais como o Bootstrap), torna-se fácil construir aplicações que podem ser acessadas em dispositivos com

diferentes resoluções. Assim, o acesso ao sistema pode ser realizado de forma simples e confortável, uma vez que necessita apenas do uso de um dispositivo conectado à internet.

Outro ponto que justifica a escolha de um sistema *web* corresponde exatamente a utilização da *internet,* uma vez que a utilização desse recurso viabiliza a utilização de sistemas computacionais ao possibilitar a manipulação de dados a partir de qualquer lugar, porquanto é um recurso "*que se apresenta como uma ferramenta essencial para a propagação da informação e colaboração no campo de pesquisa*" (BENITO e LICHESKI, 2009).

No que diz respeito efetivamente ao desenvolvimento, em relação a codificação, foi utilizado o paradigma de Programação Orientada a Objetos, haja vista que, como aponta Nascimento (2005), a utilização da POO supre as necessidades de gerenciamento de programas que possuem um elevado grau de complexidade, possuindo várias características que facilitam a reutilização e a manutenção dos *softwares* produzidos.

Quando se trabalha com POO se tem um código mais organizado e tal característica facilita o processo de compreensão e assimilação do *Back-end* que foi construído, o que permite maior ganho de produtividade durante a programação e, claro, a grande manutenibilidade que o sistema adquire, em outras palavras, um sistema cujo código não exige processos complexos para manter sua manutenção, mesmo se esta for realizada por desenvolvedores que não fizeram parte da construção do *software*.

A tecnologia principal utilizada para construção do *Back-end* foi o *framework* Laravel, cujo conceito já foi explicado anteriormente. Optou-se por essa tecnologia por ser um poderoso *framework* capaz de prover ferramentas que facilitam o processo de desenvolvimento e, dentre outros motivos, possuir uma vasta documentação de conteúdo e uma comunidade ativa, o que facilita a busca de informações, aliado ao fato do Laravel ser um *framework* muito completo e que se mantém sempre atualizado. Ainda sob esse aspecto, vale salientar que, como aponta ARÊAS(2011), "*Com certeza, economiza-se muito trabalho ao adotar um bom framework*". Observa-se ainda que por se tratar de um *framework* gratuito, acaba por se tornar um candidato natural a ser utilizado neste trabalho.

Toda a estrutura do Laravel é fundamentada no modelo MVC, portanto,

naturalmente, o sistema desenvolvido também é estruturado dessa forma e com essa separação em camadas torna-se mais fácil a manutenção do código e a compressão do processo de requisição cliente/servidor. Vale ainda considerar que "*Frameworks no estilo Model-View-Controler (MVC) são úteis para acelerar e melhorar o desenvolvimento de software.*" (PELLIZA, BERTOLINI e SILVEIRA, 2018), além de propiciar um código mais limpo e legível.

Para realização do processo de *Login* do sistema, foram utilizados o pacote *Socialite*, provido pelo Laravel, assim o processo de iniciar uma sessão no sistema pôde ter sua construção simplificada e realizada por meios seguros e eficazes.

O *Socialite* foi utilizado para que seja possível a realização de *login* por meio de perfis de clientes que fazem uso do provedor *OAuth* para acesso via Google Conta, de modo a tornar mais confortável a utilização do sistema, pois dessa forma não é necessário acessar o formulário de cadastro. Vale salientar que a decisão de se registrar diretamente no sistema cabe apenas ao usuário, assim, caso não queira acessar o sistema por meio de uma conta do Google, basta apenas se registrar por meio do formulário de cadastro, considerando, claro, a necessidade de utilização de um endereço de e-mail válido e ativo.

Para eventos *online* o sistema incorpora em sua estrutura a transmissão de *Live Stream* originada na plataforma YouTube, uma plataforma de compartilhamento de vídeos. Optou-se por trabalhar com esse recurso pois não exige grande poder de processamento, uma vez que a *stream* em si estará hospedada nos servidores do YouTube e apenas transmitida dentro do sistema. A utilização do Youtube se mostra viável por ser uma plataforma já conhecida e que já vem sendo utilizada pelo *campus* Almenara. Observa-se que dentre as soluções atuais utilizadas pela instituição, já é possível a transmissão de *stream*, porém esse recurso não garante o controle acesso e a validação de presença. Dessa forma a construção de uma ferramenta própria tratando tais apontamentos se mostrou mais viável e necessária.

O tópico a seguir aborda as tecnologias utilizadas para construção do *front-end* do sistema e foi dividido em quatro grupos distintos, baseado nas especificações de cada tecnologia e necessidade da aplicação.

5.2.2 Construção do *Front-end*

Para a construção do *Front-end* pode-se dividir sua feitura a partir das funções desempenhadas por cada tecnologia, dessa forma serão abordados os seguintes processos: Estruturação, Dinamismo, Estilização e Performance.

Para o processo de Estruturação foi utilizada a linguagem HTML. O HTML é uma linguagem base para estruturação das páginas no ambiente web, então sua utilização se torna indispensável.

Embora seja utilizado o HTML para estruturar as páginas, por vezes torna-se necessário que essas tenham um funcionamento dinâmico, ou seja, sua estrutura de conteúdo, ou até mesmo requisições ao servidor, devem ser realizadas sem que seja preciso recarregar a página ativa. Para tal, foi utilizada a linguagem de programação Javascript (JS), que, assim como o PHP, consiste em uma linguagem de script, mas diferencia-se por poder ser utilizada tanto no lado do Servidor quanto no lado do Cliente, uma vez que o Javascript é interpretado diretamente pelo navegador.

Para dar o estilo das páginas utilizou-se a linguagem CSS em sua forma nativa e o *framework* Bootstrap. Optou-se por sua utilização nativa devido às particularidades que cada página ou elemento pode vir a ter, assim, cada um pode ser editado de forma individual de acordo com as suas necessidades. Por outro lado, o Bootstrap foi utilizado devido ao ganho de produtividade que o mesmo proporciona, uma vez que, ao se fazer uso de suas classes pré-definidas muitas configurações manuais tornam-se desnecessárias, tais como configurações de responsividade. Além, claro, de todas as configurações de estilo que o próprio *framework* entrega.

Para aumentar a performance do sistema foi utilizado o *Blade Template,* tecnologia fornecida pelo Laravel. O Blade consiste de um poderoso “motor” de templates que permite a criação de Views complexas compostas por componentes que podem ser utilizados como herança entre si, sem a utilização direta do PHP.

Quando se utiliza o Blade, as Views escritas são renderizadas e armazenadas

em cache, o que elimina a presença de *overhead* nas *Views* do sistema. Vale ainda salientar que com a utilização do Blade, ainda é possível a utilização de código PHP nativo dentro das *Views*, sendo esse mais um diferencial do Blade Template.

O próximo tópico define as tecnologias utilizadas para construção e funcionamento do banco de dados.

5.2.3 Banco de Dados

O banco de dados do sistema consiste em um banco de dados relacional construído utilizando o SGBD(Sistema Gerenciador de Banco de Dados) MySQL. Optou-se pela utilização do MySQL por se tratar de um SGBD que melhor se comunica com a linguagem PHP e por sua grande utilização no mercado. Aliado ao MySQL foi utilizada a interface phpMyAdmin, por ser um *software* livre e de fácil acesso.

Ao se trabalhar com o Laravel a manipulação do banco de dados é feita por meio do ORM Eloquent, que traz maior facilidade e elegância para a interação com o banco, devido ao poderoso *Query Builder* presente no mesmo. Além disso, o Eloquent transforma as tabelas do banco em classes, chamadas de Modelo, e esses modelos são perfeitos para utilização com a arquitetura MVC. Isto significa que para cada tabela no banco, há um modelo correspondente no *Back-end.*

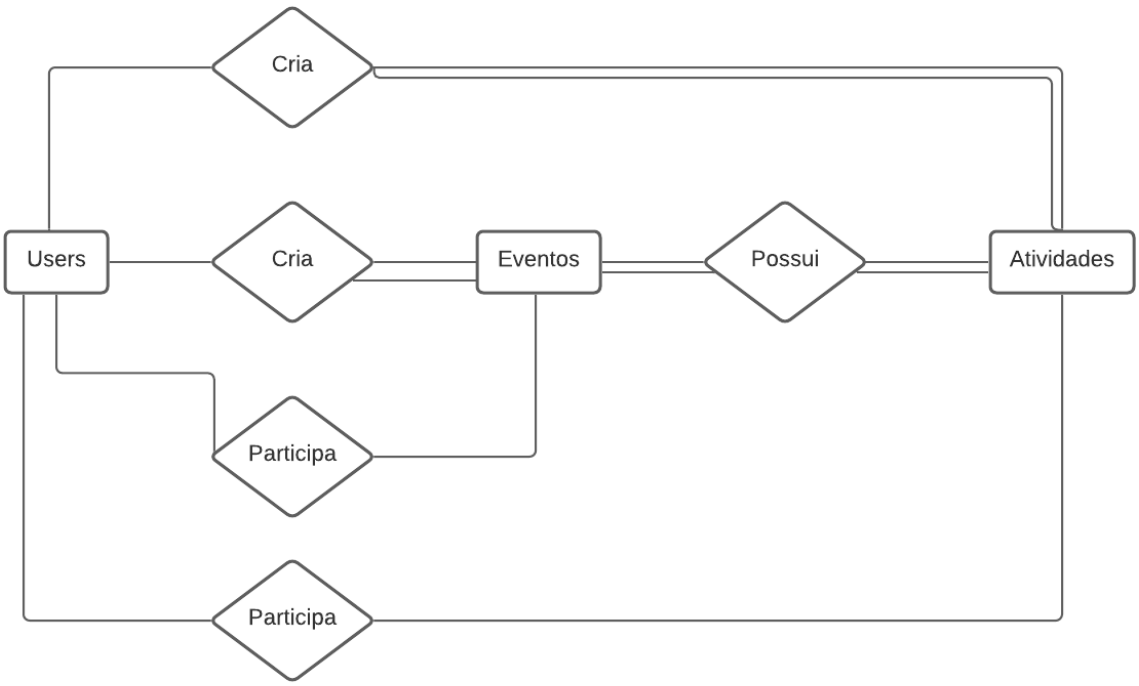
Por meio do Eloquent é possível realizar toda a persistência de dados da aplicação, o que significa cadastrar, consultar, editar e excluir dados. Todo esse processo é feito por meio de métodos pré-definidos no ORM, dessa forma dispensa a necessidade de se escrever códigos SQL nativos, uma vez que, por meio do *Query Builder* do Eloquent o SQL é gerado automaticamente. Ainda assim, vale salientar que também é possível manipular o banco de dados usando *queries* escritas diretamente em SQL.

Outra vantagem é o versionamento do banco, pois toda a construção e alteração na estrutura do banco de dados é feita por meio de *migrations,* com isso é possível saber exatamente quando uma mudança foi implementada e dessa forma pode ser facilmente revertida.

Na Figura 2 é possível observar o modelo relacional inicial proposto para o banco de dados do sistema. Esse modelo foi apenas um protótipo inicial do sistema,

pois uma vez que o *software* foi desenvolvido seguindo o modelo incremental, novos requisitos foram surgindo durante o desenvolvimento do sistema, desta forma por escolha do autor e em conformidade com alguns dos conceitos da *eXtreme Programming,* optou-se pela não definição de um modelo prévio com todos os requisitos, assim o banco pode ser moldado de acordo as suas necessidades. Considera-se ainda que esse modelo se apresenta como a estrutura básica de um sistema de gestão de eventos.

Figura 2: Protótipo inicial do Modelo Relacional do banco de dados



Fonte: Autor

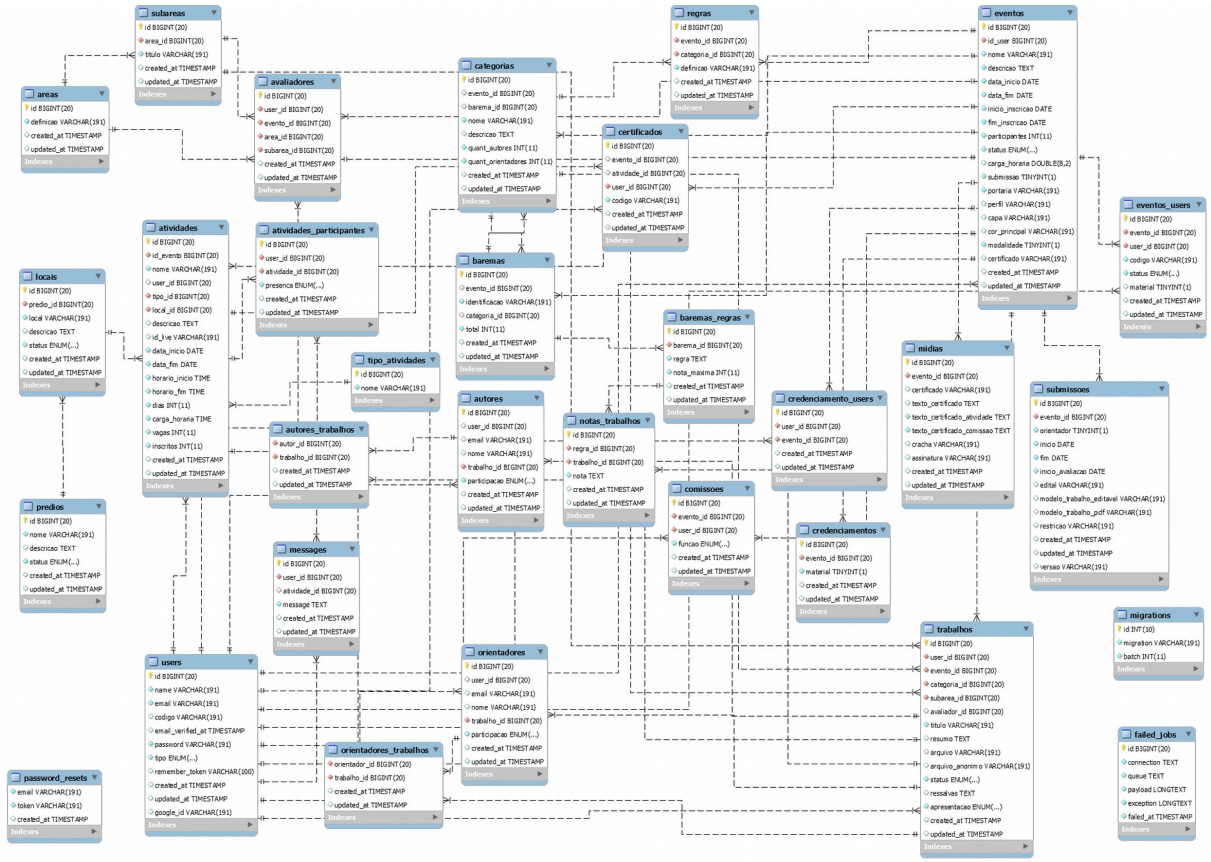
No modelo proposto, destaca-se que:

- 1 usuário pode cadastrar N eventos,
- 1 evento possui N atividades,
- 1 atividade pertence a 1 evento

- N usuários participam (ou não) de N eventos
- N usuários participam (ou não) de N atividades

Na figura 3 é possível observar o modelo relacional do banco de dados após a composição de todas as tabelas.

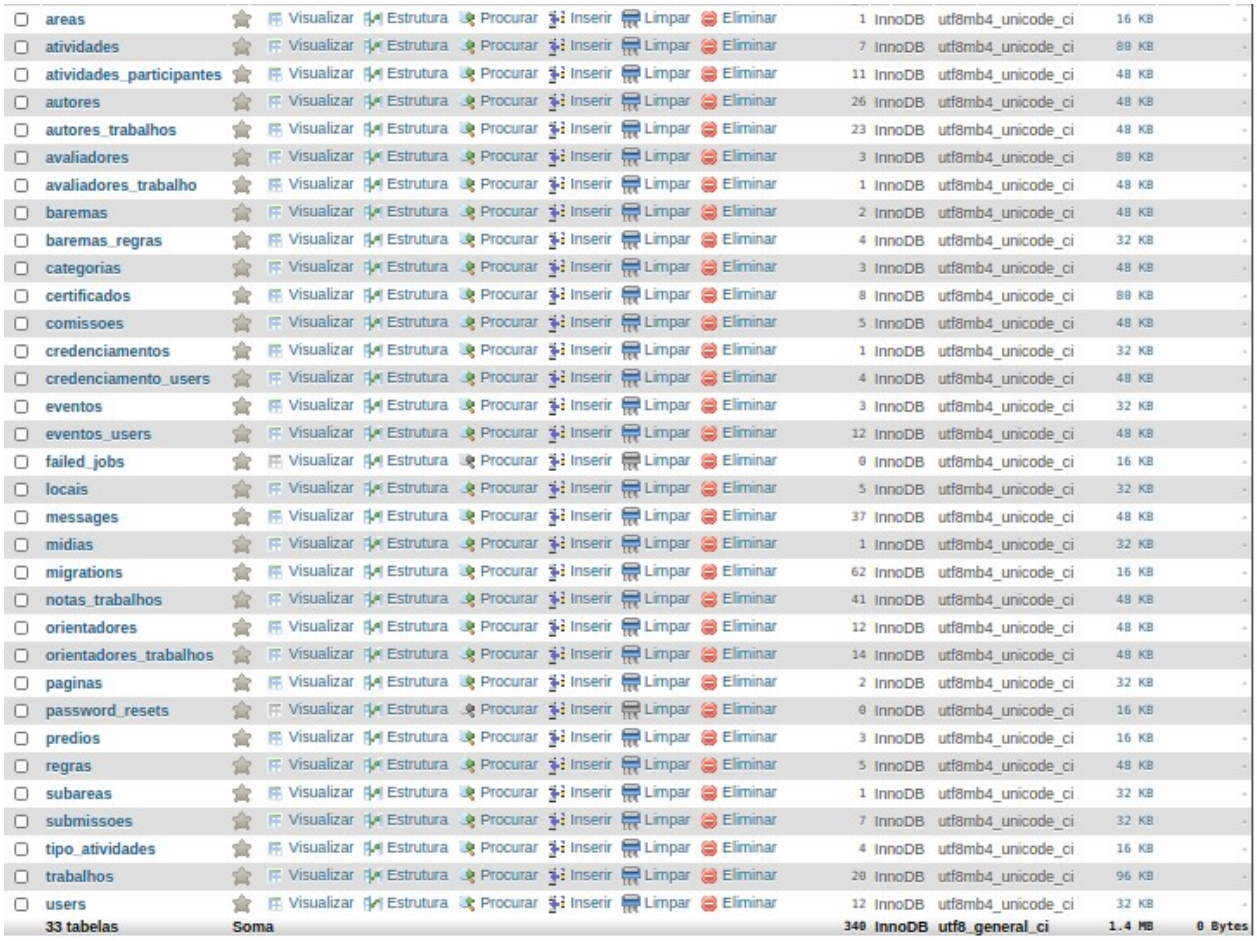
Figura 3: Modelo relacional completo do Banco de Dados



Fonte: Autor

Na figura seguinte é possível observar toda a estrutura final de tabelas do banco de dados do sistema.

Figura 3: Estrutura de tabelas do banco de dados



areas	Visualizar	Estrutura	Procurar	Inserir	Limpar	Eliminar	1	InnoDB	utf8mb4_unicode_ci	16 KB	
atividades	Visualizar	Estrutura	Procurar	Inserir	Limpar	Eliminar	7	InnoDB	utf8mb4_unicode_ci	80 KB	
atividades_participantes	Visualizar	Estrutura	Procurar	Inserir	Limpar	Eliminar	11	InnoDB	utf8mb4_unicode_ci	48 KB	
autores	Visualizar	Estrutura	Procurar	Inserir	Limpar	Eliminar	26	InnoDB	utf8mb4_unicode_ci	48 KB	
autores_trabalhos	Visualizar	Estrutura	Procurar	Inserir	Limpar	Eliminar	23	InnoDB	utf8mb4_unicode_ci	48 KB	
avaliadores	Visualizar	Estrutura	Procurar	Inserir	Limpar	Eliminar	3	InnoDB	utf8mb4_unicode_ci	80 KB	
avaliadores_trabalho	Visualizar	Estrutura	Procurar	Inserir	Limpar	Eliminar	1	InnoDB	utf8mb4_unicode_ci	48 KB	
baremas	Visualizar	Estrutura	Procurar	Inserir	Limpar	Eliminar	2	InnoDB	utf8mb4_unicode_ci	48 KB	
baremas_regras	Visualizar	Estrutura	Procurar	Inserir	Limpar	Eliminar	4	InnoDB	utf8mb4_unicode_ci	32 KB	
categorias	Visualizar	Estrutura	Procurar	Inserir	Limpar	Eliminar	3	InnoDB	utf8mb4_unicode_ci	48 KB	
certificados	Visualizar	Estrutura	Procurar	Inserir	Limpar	Eliminar	8	InnoDB	utf8mb4_unicode_ci	80 KB	
comissoes	Visualizar	Estrutura	Procurar	Inserir	Limpar	Eliminar	5	InnoDB	utf8mb4_unicode_ci	48 KB	
credenciamentos	Visualizar	Estrutura	Procurar	Inserir	Limpar	Eliminar	1	InnoDB	utf8mb4_unicode_ci	32 KB	
credenciamento_users	Visualizar	Estrutura	Procurar	Inserir	Limpar	Eliminar	4	InnoDB	utf8mb4_unicode_ci	48 KB	
eventos	Visualizar	Estrutura	Procurar	Inserir	Limpar	Eliminar	3	InnoDB	utf8mb4_unicode_ci	32 KB	
eventos_users	Visualizar	Estrutura	Procurar	Inserir	Limpar	Eliminar	12	InnoDB	utf8mb4_unicode_ci	48 KB	
failed_jobs	Visualizar	Estrutura	Procurar	Inserir	Limpar	Eliminar	0	InnoDB	utf8mb4_unicode_ci	16 KB	
locais	Visualizar	Estrutura	Procurar	Inserir	Limpar	Eliminar	5	InnoDB	utf8mb4_unicode_ci	32 KB	
messages	Visualizar	Estrutura	Procurar	Inserir	Limpar	Eliminar	37	InnoDB	utf8mb4_unicode_ci	48 KB	
midias	Visualizar	Estrutura	Procurar	Inserir	Limpar	Eliminar	1	InnoDB	utf8mb4_unicode_ci	32 KB	
migrations	Visualizar	Estrutura	Procurar	Inserir	Limpar	Eliminar	62	InnoDB	utf8mb4_unicode_ci	16 KB	
notas_trabalhos	Visualizar	Estrutura	Procurar	Inserir	Limpar	Eliminar	41	InnoDB	utf8mb4_unicode_ci	48 KB	
orientadores	Visualizar	Estrutura	Procurar	Inserir	Limpar	Eliminar	12	InnoDB	utf8mb4_unicode_ci	48 KB	
orientadores_trabalhos	Visualizar	Estrutura	Procurar	Inserir	Limpar	Eliminar	14	InnoDB	utf8mb4_unicode_ci	48 KB	
paginas	Visualizar	Estrutura	Procurar	Inserir	Limpar	Eliminar	2	InnoDB	utf8mb4_unicode_ci	32 KB	
password_resets	Visualizar	Estrutura	Procurar	Inserir	Limpar	Eliminar	0	InnoDB	utf8mb4_unicode_ci	16 KB	
predios	Visualizar	Estrutura	Procurar	Inserir	Limpar	Eliminar	3	InnoDB	utf8mb4_unicode_ci	16 KB	
regras	Visualizar	Estrutura	Procurar	Inserir	Limpar	Eliminar	5	InnoDB	utf8mb4_unicode_ci	48 KB	
subareas	Visualizar	Estrutura	Procurar	Inserir	Limpar	Eliminar	1	InnoDB	utf8mb4_unicode_ci	32 KB	
submissoes	Visualizar	Estrutura	Procurar	Inserir	Limpar	Eliminar	7	InnoDB	utf8mb4_unicode_ci	32 KB	
tipo_atividades	Visualizar	Estrutura	Procurar	Inserir	Limpar	Eliminar	4	InnoDB	utf8mb4_unicode_ci	16 KB	
trabalhos	Visualizar	Estrutura	Procurar	Inserir	Limpar	Eliminar	20	InnoDB	utf8mb4_unicode_ci	96 KB	
users	Visualizar	Estrutura	Procurar	Inserir	Limpar	Eliminar	12	InnoDB	utf8mb4_unicode_ci	32 KB	
33 tabelas	**Soma**						**340**	**InnoDB**	**utf8_general_ci**	**1.4 MB**	**0 Bytes**

Fonte: Autor

O tópico “Versionamento” apresenta as tecnologias utilizadas para realizar, efetivamente, o controle de versões do sistema, além de permitir o fácil acesso ao código fonte (salientando que não se trata de um sistema de código aberto, mas que poderá vir a ser desenvolvido por outros desenvolvedores) e o *backup*, caso seja necessário.

5.2.4 Versionamento

Para realizar o versionamento do sistema, utilizou-se o sistema Git. O Git, como apresentado no item 4.12 do Referencial Teórico, consiste em um sistema que possibilita o compartilhamento e versionamento de projetos. Observa-se ainda que o Git armazena quase todas as suas informações de maneira local, o que garante que o desempenho de comunicação em uma rede não seja afetado por problemas de

latência, conforme Barbosa e Da Silva (2018).

Para garantir o versionamento e *backup* do sistema, foram utilizados o sistema de versionamento Git e, para hospedagem, o sistema de repositórios Github. Como outras alternativas podem ser citados o GitLab e o Bitbucket, que também são plataformas de hospedagem de repositórios Git. Optou-se pela utilização do Github por se tratar de uma plataforma de uso gratuito e que possui maior engajamento entre a comunidade de desenvolvedores, concordante ao fato de que o Github, dentre as plataforma de hospedagem de repositórios, acaba "*Sendo o maior host único para repositórios Git, é o ponto central de colaboração de milhões de desenvolvedores e projetos."(BARBOSA e DA SILVA,* 2018).

Nesse ponto deve ser observado a necessidade de utilização de um sistema/ plataforma de versionamento dos códigos do sistema, pois sabe-se que posteriormente esse *software* deverá ser mantido por outros desenvolvedores, então com a utilização de um sistema de hospedagem aumenta-se a facilidade em disponibilizar o código fonte do *software,* pois consoante a Bosse (2019), o usuário pode compartilhar e conceder acesso a outras pessoas para que elas possam colaborar com o projeto. Além disso, com o Github é possível manter o histórico de todas as atualizações que foram aplicadas no sistema desde a criação das primeiras linhas de código. Aliada a isso, pensando no uso do Github e Git, Konnorate *et al* (2019) aponta que "*o Git é um dos (se não o mais) usado software para versionamento de arquivos atualmente. Um recurso essencial para o desenvolvimento de softwares.*".

6. RESULTADOS E DISCUSSÕES

O *software* desenvolvido é um sistema *web* desenvolvido com o Framework Laravel. Por se tratar de um sistema *web* pode ser acessado por qualquer navegador, entretanto, recomenda-se a utilização do navegador Google Chrome.

O *software* foi desenvolvido utilizando em seu *front-end*, dentre outras tecnologias, o *framework* Bootstrap, assim todo o sistema é responsivo. Portanto, pode ser acessado por qualquer dispositivo que disponha de um navegador web.

O sistema possui uma estrutura voltada para obedecer 3 níveis de acesso, tendo os privilégios e acesso a funcionalidades definidos por meio desses níveis, sendo eles:

- Padrão, nível mais baixo de acesso. Nível reservado especialmente para participantes, usuários que apenas irão participar de algum evento.
- Administrador, nível intermediário. Destinado aos usuários com privilégios dentro do sistema, podendo criar e gerenciar eventos, bem como todas as configurações relacionadas aos mesmos.
- Super Administrador, nível mais elevado. Nível destinado ao gerente do sistema. O Super Administrador possui todas as funções dos níveis anteriores, acrescida de funcionalidades administrativas, tais como: aprovar ou reprovar eventos e definir níveis de acesso para outros usuários.

6.1 Das funcionalidades

Nesta seção serão apresentadas as funcionalidades (Considerando os diferentes níveis de acesso que o sistema oferece) que estão disponíveis no sistema. Vale ainda salientar que serão abordadas as funcionalidades essenciais do sistema, sendo assim, os itens a seguir não devem ser entendidos como todos os requisitos contemplados.

Do item 1 ao item 3 são correspondentes a funcionalidades que estão disponíveis para qualquer usuário do sistema, mesmo aqueles não autenticados.

1. Possibilitar ao usuário se cadastrar e acessar o sistema. Por definição todo novo usuário é classificado com o nível de acesso “Padrão”.
2. Apresentar uma tela de navegação que permite ao utilizador receber informações sobre todos os eventos realizados, inclusive consultar eventos já encerrados.
3. Permitir ao utilizador verificar a autenticidade de certificados.

Do item 4 ao item 8 são correspondentes a funcionalidades que estão disponíveis para qualquer nível de acesso, entretanto, são disponíveis apenas para usuários autenticados no sistema.

4. Permitir aos usuários se inscreverem nos eventos cadastrados no sistema. Essa funcionalidade está disponível para qualquer usuário registrado no sistema, entretanto é necessário estar “logado” no sistema para realização da inscrição.
5. Permitir ao usuário acessar o painel do evento, onde obterá todas as informações relacionadas a sua inscrição em um evento específico. É neste painel que o usuário pode gerenciar suas inscrições em atividades, imprimir certificados e comprovante de inscrição, bem como fazer o cancelamento de sua inscrição no evento em questão. Essa funcionalidade também está disponível em todos os níveis de acesso.
6. Permitir ao usuário se inscrever nas atividades do evento, bem como cancelar sua inscrição.
7. Permitir ao usuário realizar o credenciamento do participantes. Essa funcionalidade é atribuída pela comissão organizadora do Evento.
8. Permitir ao usuário realizar a avaliação de trabalhos submetidos.

Do item 9 ao item 11 correspondem a funcionalidades disponíveis apenas para usuários com acesso de Administrador e Super Administrador.

9. Permitir o cadastro e toda a manutenção dos eventos. Entende-se aqui como manutenção a capacidade de editar os dados do evento e acompanhar as demais informações, tais como: número de participantes inscritos, participantes presentes, detalhes sobre trabalhos submetidos (caso haja a submissão de trabalhos). Pode se categorizar ainda nesse item: o cadastro dos membros da comissão organizadora, cadastro dos membros da equipe de credenciamento,

aprovação e cadastro dos avaliadores de trabalhos, definição dos textos dos certificados, anexo de arquivos, impressão de arquivos e envio de mensagens para os participantes.

10. Permitir a exclusão de um evento
11. Permitir ao utilizador gerenciar todas as configurações relativas a submissão de trabalho.
12. Permitir ao usuário realizar o credenciamento manual dos participantes em atividades.

A partir do item 12 as funcionalidades estão disponíveis apenas para usuários com acesso de Super Administrador.

13. Permitir ao usuário cadastrar e realizar a manutenção dos locais e ambientes. Entende-se, aqui, por manutenção: editar os dados, ativar, inativar e excluir.
14. Permitir ao usuário listar todos os demais usuários.
15. Permitir ao usuário a manutenção de outros usuários. Aqui a manutenção pode ser entendida como: excluir um usuário e mudar o nível de acesso de outros usuários.
16. Permitir ao usuário a listagem de todos os eventos cadastrados no sistema.
17. Permitir ao usuário aprovar ou reprovar a realização de um evento.

O software desenvolvido ainda é capaz de realizar eventos de forma inteiramente *online.* Dessa forma, para eventos que assim se classificam, o sistema fornece todas as funcionalidades anteriormente citadas e ainda é capaz de permitir ao participante acompanhar as atividades por meio de *Live Stream,* em outras palavras, a transmissão de dados em tempo real por meio da internet.

Ainda sobre as funcionalidades, o sistema desenvolvido é capaz de gerar os arquivos de mídia necessários para os eventos, tais como certificados e crachás. Os certificados são gerados de forma automática e mantidos por tempo indeterminado, o que permite ao participante recuperar informações com facilidade a qualquer momento. Salienta-se aqui a importância de se ter acesso a informação de forma facilitada, uma vez que se deve levar em consideração o valor que a informação quando adequadamente assimilada possui, haja vista que:

> A Informação assume, hoje em dia, uma importância crescente. Ela torna-se fundamental a nível da empresa na descoberta e introdução de novas tecnologias, exploração das oportunidades de investimento e ainda na planificação de toda a actividade industrial" (BRAGA, 2000).

O sistema ainda apresenta um tradutor de Libras (Linguagem Brasileira de Sinais), disponibilizado de forma gratuita pelo governo federal, por meio de um *script* externo que foi adicionado ao código fonte do sistema. Esse mecanismo foi adicionado com o intuito de facilitar e tornar o sistema mais acessível. Tal funcionalidade se apresenta como indispensável, pois:

> A globalização do acesso a informações requer uma atenção especial nos projetos de aplicações Web, que se depara com o desafio de atender a diferentes necessidades e restrições dos usuários.(FREIRE, 2008).

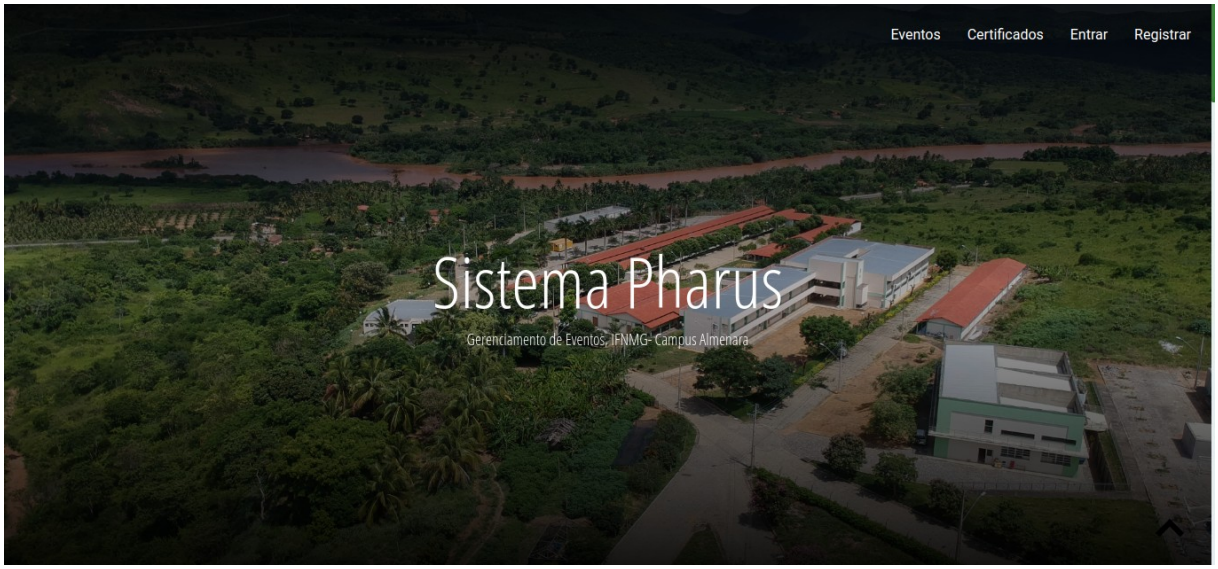
Assim sendo, todas as telas do sistema podem ter seu conteúdo textual apresentado em forma de Libras, por meio de animações. Para tal, utilizou-se o VLibras, que nada mais é que "*um conjunto de ferramentas computacionais de código aberto, que traduz conteúdos digitais (texto, áudio e vídeo) para Língua Brasileira de Sinais - LIBRAS"* (VLibras, 2021).

6.2 Telas

A seguir serão apresentadas algumas telas do sistema, de acordo com o nível de acesso. Vale lembrar que essas telas e as informações contidas nas mesmas podem já ter sido atualizadas ou sofrido alguma modificação. Para construção das telas não foi realizada prototipação prévia e sua feitura foi realizada seguindo os conceitos da Interação Homem Máquina, abordados durante o curso de Análise e Desenvolvimento de Sistemas, de modo a tornar mais confortável, intuitiva e acessível a utilização do *software*.

- Tela inicial do sistema, apresentada na figura 4. Essa tela é apresentada a qualquer usuário que acesse o sistema. É dividida em 3 sessões, sendo ela: Início, Eventos e Certificados. Além dessas sessões, possuem também os botões de acesso aos formulários de *login* e cadastro. As imagens utilizadas na página inicial do sistema são imagens do *campus* Almenara, disponibilizadas gratuitamente por um servidor.

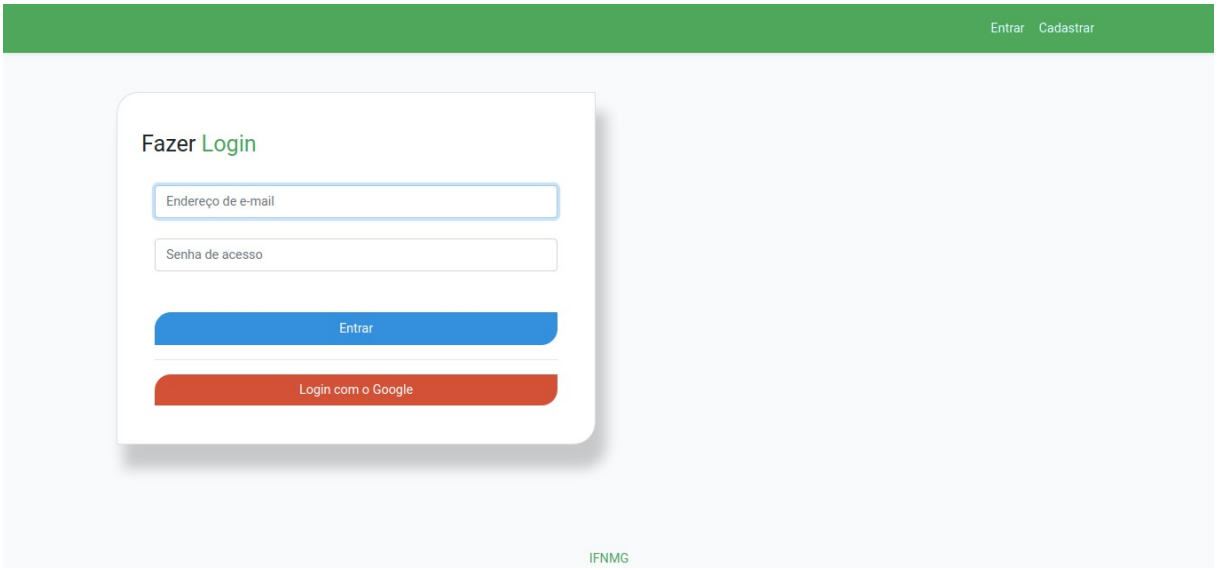
Figura 4, tela inicial do sistema.



Fonte: Autor

- Tela que apresenta o formulário de *Login* do sistema, Figura 5. A tela de *login* é a mesma independente do nível de acesso. Nessa tela são apresentados 4 campos no formulário, caso o usuário queira logar por meio de um perfil previamente criado diretamente no sistema, basta preencher os campos de e-mail e senha e clicar em entrar. Caso opte por entrar utilizando uma conta do Google, basta clicar sobre o botão “Login com o Google” e selecionar o perfil desejado. Caso o usuário ainda não tenha cadastro, basta clicar sobre o *link* “Cadastrar” presente no canto superior direito.

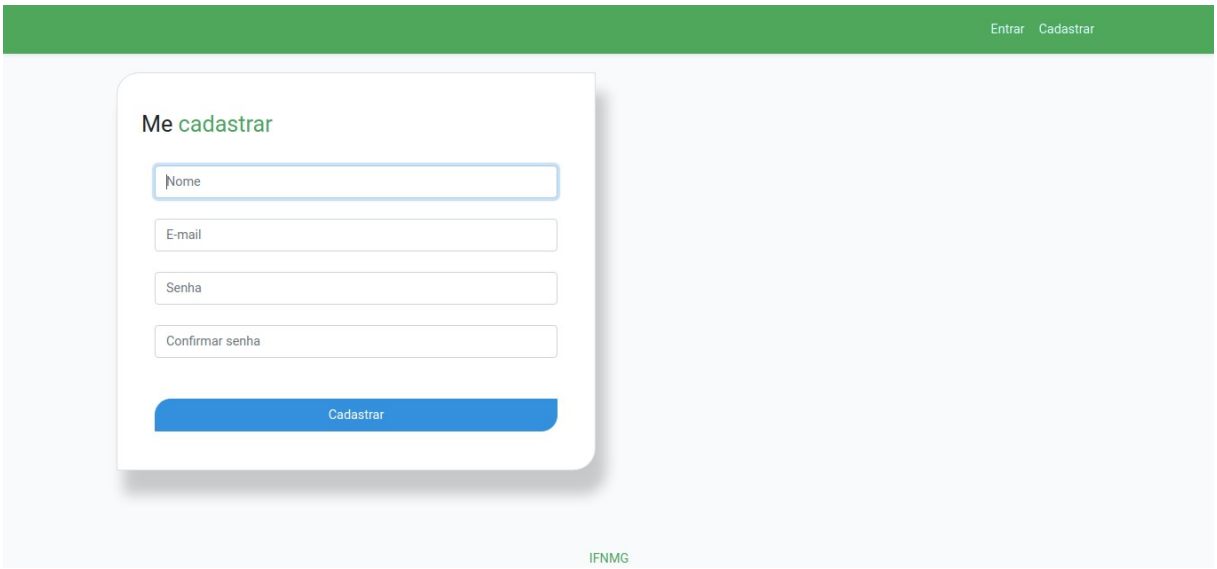
Figura 5. Tela de login do sistema



Fonte: Autor

- Tela que apresenta o formulário de cadastro do sistema, Figura 6.

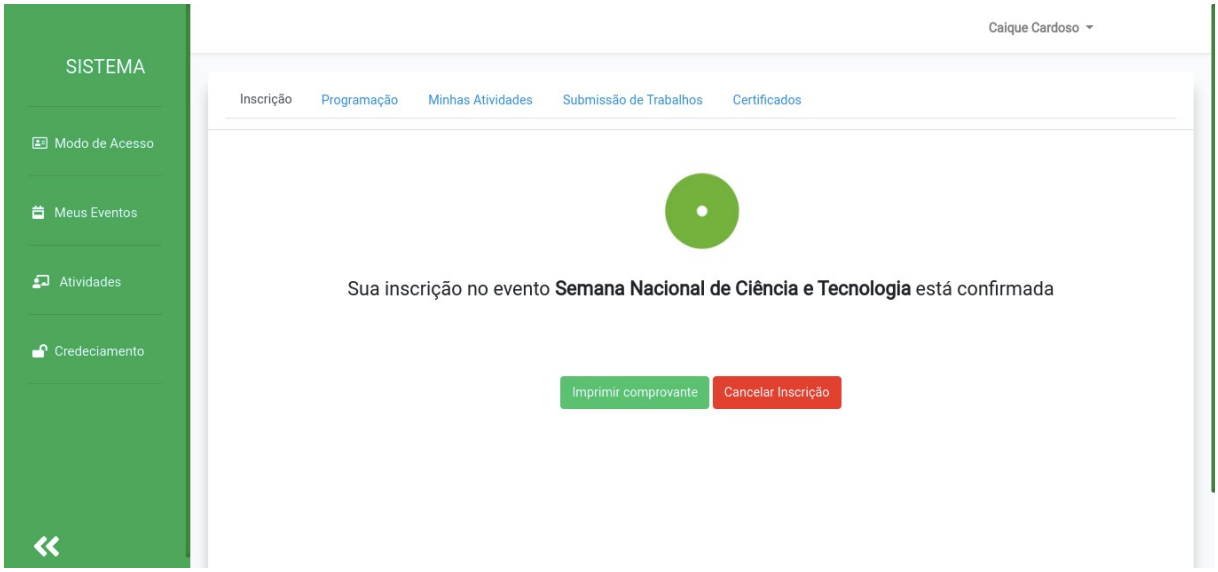
Figura 6. Tela de cadastro do sistema



Fonte: Autor

- Tela que apresenta o painel do evento. Essa tela é destinada aos participantes, assim, podem realizar aqui toda a gestão de sua participação.

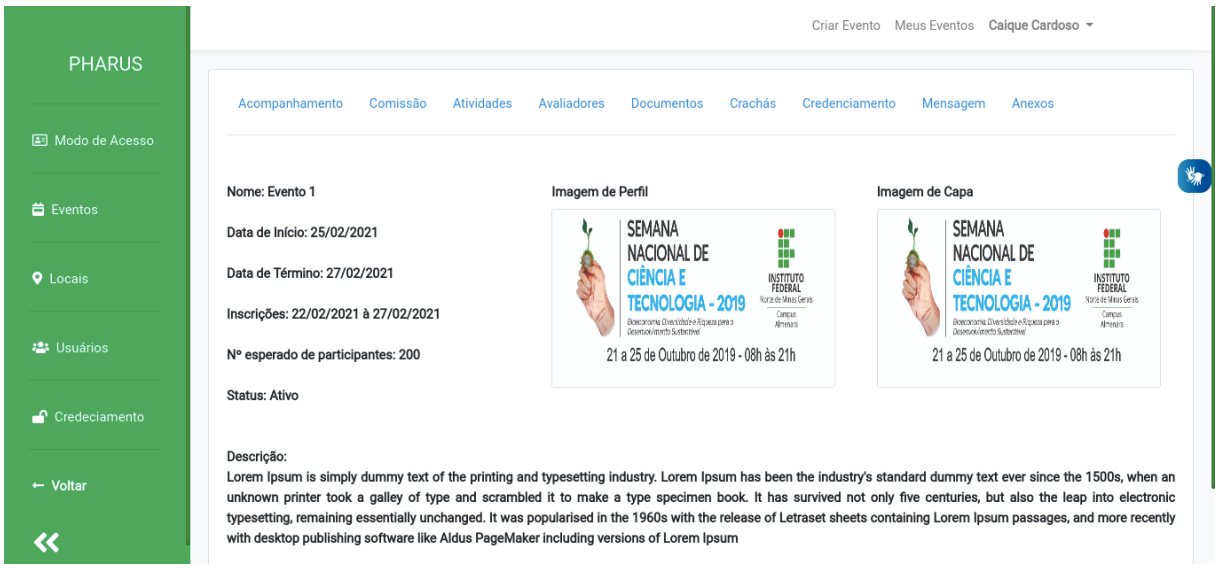
Figura 7. Painel do evento para os participantes



Fonte: Autor

- Tela que apresenta o painel de gerência do evento. Destinada somente para usuários autenticados

Figura 8. Painel de gerência do evento.

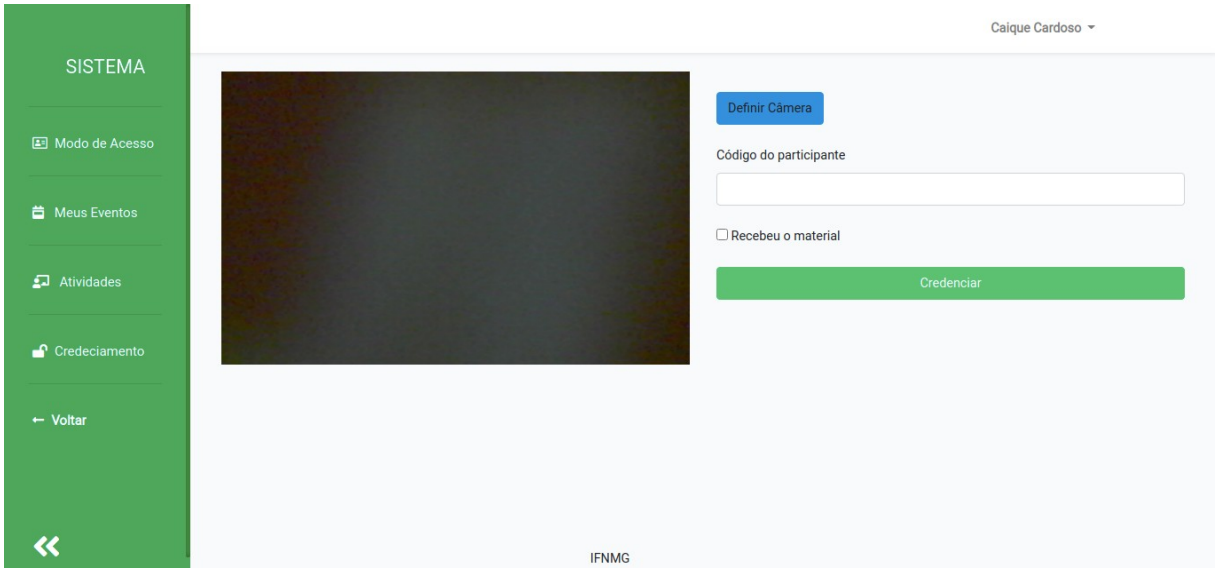


Fonte: Autor

- Tela que apresenta o formulário de credenciamento. O credenciamento dentro

do sistema pode ser feito por meio da leitura do *QR Code* presente no crachá e no comprovante de inscrição. Pode ser feito ainda de forma manual inserindo o código de participação do participante.

Figura 9: Tela de credenciamento do sistema



Fonte: Autor

7. TRABALHOS SEMELHANTES

Para que o software pudesse ser construído, optou-se por consultar e realizar o levantamento de soluções análogas. Por se tratar de uma demanda recorrente em ambientes educacionais de nível superior, alguns trabalhos já foram desenvolvidos e o levantamento realizado visou compreender os cenários possíveis de aplicabilidade e demanda que os *softwares* propostos eram capazes de suprir. Dessa forma, realizar um estudo de tecnologias que poderiam ser utilizadas para construção do sistema e absorver outras demandas que deveriam ser atendidas.

O Academeek (ROCHA *et al.,* 2016) é um exemplo de sistema desenvolvido para gestão de eventos estudantis. Este *software* é capaz de realizar o controle de presença, emitir certificados e crachás, planejar e gerir os eventos.

O e-Vent-Br (DA SILVA CARVALHO *et al.,* 2014) é um sistema gerenciador de eventos acadêmicos, também feito para *web*, publicado sob uma licença *open source.* Tem por objetivo gerenciar as informações relativas aos eventos.

(MARTINS FILHO; GLANZMANN, 2018) apresentem seu trabalho intitulado “UM SISTEMA PARA GERÊNCIA E MANUTENÇÃO DE EVENTOS ACADÊMICOS”, um sistema que está apto a criar e gerir eventos acadêmicos em um contexto web. O sistema foi desenvolvido usando a linguagem PHP e o framework Laravel.

O S-EVA, consoante (LIMA, 2016), é um protótipo de sistema que implementa as soluções propostas para a realização de eventos acadêmicos, se pondo como uma ferramenta de auxílio.

O sistema Even3. Além dos trabalhos citados, há ainda o *software* Even3. Esse sistema é mantido pela empresa L3 Soluções em Tecnologia, podendo ser gratuito ou pago a depender da intenção do utilizador e da quantidade de participantes que o evento terá, caso exceda o limite permitido em sua versão gratuita, um valor proporcional ao número de inscritos excedido é cobrado.

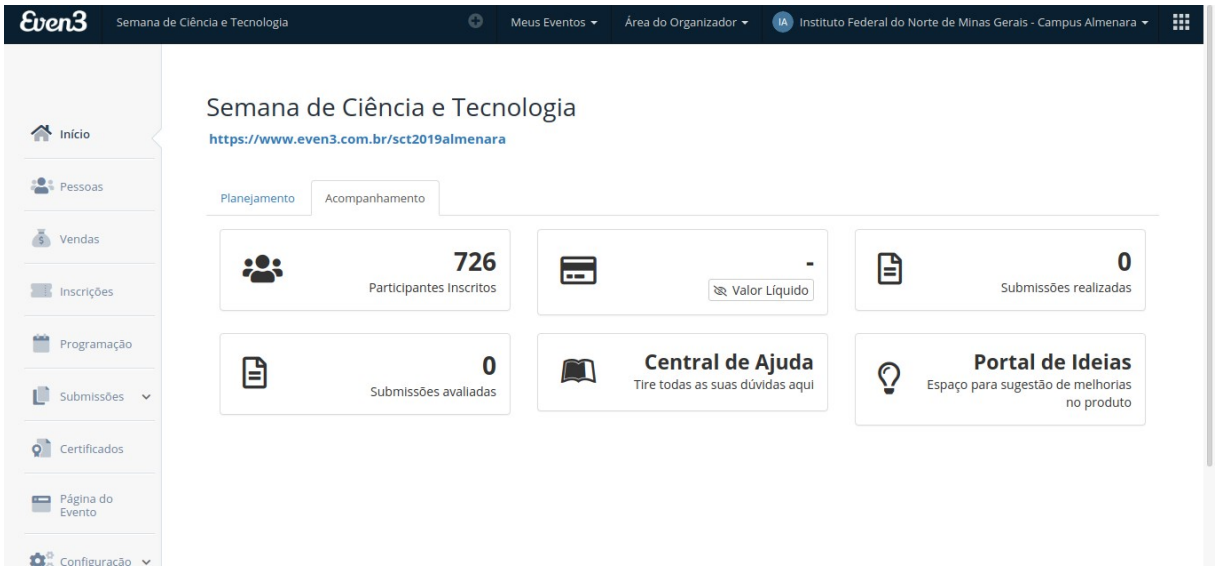
O Even3 é um sistema *web* que dentre suas funcionalidades estão:

- Gerenciar as inscrições
- Manutenção de atividades
- Gerência de certificados que são gerados automaticamente
- Prover formulário de avaliação do Evento

- Manutenção de trabalhos

Na figura a seguir é apresentada a tela inicial do Even após efetuar *login.*

Figura 10. Tela inicial do Even3, onde é possível gerenciar um evento cadastrado.



Fonte: Autor, captura de tela do sistema Even3

8. CONSIDERAÇÕES FINAIS

Com o fim das atividades propostas, constata-se que todos os objetivos previamente definidos foram concluídos. Obteve-se um sistema seguro capaz de gerenciar as demandas que são exigidas pela Instituição, podendo ser utilizado para gerenciar eventos de grande, médio e pequeno porte. Salienta-se ainda a capacidade que o sistema possui de evoluir e anexar em suas estrutura novas funcionalidades. O sistema possibilita que instituições de ensino possam dispor de maior comodidade e facilidade para gerir seus eventos, assim podendo focar sua atenção em outras operações.

Vale ainda observar que um dos principais objetivos deste trabalho consiste em pôr em prática os conhecimentos adquiridos durante o curso de Análise e Desenvolvimento de Sistemas, tanto para os conhecimentos teóricos, quanto aos práticos. Assim sendo, é notório que ao se obter um software funcional, construído a partir de conceitos, metodologias e ferramentas aprendidas durante o curso, é certo constatar que esse objetivo também foi atingido.

8.1 Trabalhos futuros

O sistema foi desenvolvido buscando possuir um código de fácil compreensão e adição de novo módulos, para que seja possível prover atualizações e novas funcionalidades ao sistema. Dentre os objetivos a curto prazo, espera-se realizar o primeiro teste do sistema em ambiente de produção, realizando um evento online, para assim poder avaliar as funcionalidades já implementadas e realizar o planejamento de novas funções.

Outro objetivo é a disponibilização da documentação de uso do sistemas, permitindo maior acesso para futuros usuários da aplicação.

Espera-se ainda que a longo o prazo o sistema possa ser expandido para atender outras unidades do Instituto Federal do Norte de Minas Gerais.

REFERÊNCIAS

Aquino, E. M., Silveira, I. H., Pescarini, J. M., Aquino, R., & Souza-Filho, J. A. D. **Medidas de distanciamento social no controle da pandemia de COVID-19: potenciais impactos e desafios no Brasil**. *Ciência & Saúde Coletiva*, *25*, 2423-2446, 2020.

ARÊAS, Tayse Cascaes. Desenvolvimento de um gerador de aplicações web em PHP que utilizam os frameworks zend framework e doctrine. **Sistemas de Informação-Pedra Branca**, 2011.

ASTOLFI, Bruno; CABRINI, Fábio Henrique. **A arquitetura MVC no desenvolvimento de jogos para navegadores**. FaSCi-Tech, v. 1, n. 6, 2016.

BARBOSA, Gabriel Silva; DA SILVA, Evaldo de Oliveira. **Geração de Informações Gerenciais para Sistemas de Controle de Versão Um estudo de caso utilizado o GitHub**. Caderno de Estudos em Sistemas de Informação, v. 5, n. 1, 2018.

BARBOSA, Gustavo Tavares. **Laravel -Utilizando rotas para acessar Controllers**, 2019. Disponível em: <https://medium.com/@gtbarbosa/laravel-utilizando-rotas-para-acessar-controllers-6081cb7a012a>. Acesso em 22 de jan. de 2021.

BENITO, Gladys Amélia Véles; LICHESKI, Ana Paula. Sistemas de Informação apoiando a gestão do trabalho em saúde. Revista Brasileira de Enfermagem, v. 62, n. 3, p. 447-450, 2009.

BOSSE, Jacyara et al. Implementação de métricas ágeis integradas à plataforma GitHub. 2019.

BRAGA, Ascenção. **A gestão da informação**. Millenium, 2000.

DA SILVA CARVALHO, Luis Paulo et al. **e-Vent-Br: Proposta de um Sistema web de Gerenciamento de Eventos Acadêmicos.** 2014.

DE SOUZA, Pedro Menezes Ribeiro. **Um estudo sobre padrões e tecnologias para o desenvolvimento web-front-end**. 2016. Tese de Doutorado. Universidade Federal do Rio de Janeiro.

FREIRE, André Pimenta. **Acessibilidade no desenvolvimento de sistemas web:**

um estudo sobre o cenário brasileiro. 2008. Tese de Doutorado. Universidade de São Paulo.

Git, 2021. Disponível em: <https://git-scm.com/>. Acesso em: 22 de jan. de 2021.

KONNORATE, Gabriele Caroline et al. **A IMPORTANCIA DO CONTROLE DE VERSÕES NO DESENVOLVIMENTO DE SOFTWARE**. Seminário De Tecnologia Gestão E Educação, v. 1, n. 2, p. 1-4, 2019.

Laravel, 2021. Disponível em: <https://laravel.com/docs/7.x>. Acesso em: 22 de jan. de 2021.

LIMA, Ramon Moreira. **S-EVA: Um Sistema para Controle de Eventos Acadêmicos Baseado em um Projeto de Base de Dados Relacional.** 2016.

MARTINS FILHO, Luís Rogério Ventura; GLANZMANN, José Honório. **UM SISTEMA PARA GERÊNCIA E MANUTENÇÃO DE EVENTOS ACADÊMICOS**. Seminários de Trabalho de Conclusão de Curso do Bacharelado em Sistemas de Informação, v. 3, n. 1, 2018.

NASCIMENTO, Luiz Carlos do. **Avaliação da confiabilidade utilizando programação orientada a objeto: aplicação em um sistema elétrico interligado a uma central de cogeração**. 2005. 136 f. Dissertação (mestrado) - Universidade Estadual Paulista, Faculdade de Engenharia, 2005.

PELIZZA, Angelica Caetane; BERTOLINI, Cristiano; SILVEIRA, Sidnei Renato. **Um estudo sobre Técnicas de Teste de Software no Framework Laravel**. Revista Eletrônica de Sistemas de Informação e de Gestão Tecnológica, v. 9, n. 3, 2018.

PHP, 2021. Disponível em: <https://www.php.net/>. Acesso em: 22 de jan. de 2021.

PORTELA, João Luiz da Rocha. **Ergodesign e usabilidade como conceito no desenvolvimento de interfaces web com framework boostrap**. Trabalho de Conclusão de Curso (Sistemas de Informação). Antonio Meneghetti Faculdade – AMF, 2016.

RABETTI, Rafael dos Santos. **Comparativo entre as metodologias atômica e tradicional no desenvolvimento de CSS**. 2019. 48 f., il. Trabalho de Conclusão de Curso (Bacharelado em Engenharia de Software)—Universidade de Brasília, Brasília, 2019.

RICARTE, Ivan Luiz Marques. **Programação Orientada a Objetos: uma abordagem com Java**. Campina:Unicamp. 2001. Disponível em: <http://www. dca. fee. unicamp. br/cursos/PooJava/Aulas/poojava. pdf> Acesso em. 18 de jan de 2021.

ROCHA, Lucas Vasconcelos. **Academeek: gestão de eventos academicos**, Trabalho de Conclusão de Curso (Tecnologia em Análise e Desenvolvimento de Sistemas)- Universidade Federal do Paraná, 2016.

VALERIM, Patrícia; DE SOUSA, Rodrigo Silva Caxias. **Live streaming em tempos pandêmicos: práticas informacionais realizadas por profissionais da Ciência da Informação**. AtoZ: novas práticas em informação e conhecimento, v. 9, n. 2, p. 173-182, 2020.

VLibras, 2021. Disponível em: <https://www.vlibras.gov.br/>. Acesso em: 22 de jan. de 2021.

ZUBOFF, Shoshana. **Automatizar/informatizar: as duas faces da tecnologia inteligente**. Revista de administração de empresas, v. 34, n. 6, p. 80-91, 1994.